

US Road Network Analysis Using Neo4j Graph Database and Graph Data Science Algorithms

Wayne Rubangisa
Jimmy Irakiza
Lydia Kigero Mbabazi
Dany Nkurunziza

Big data Analytics - MSDA 9215
Temitope Oguntade
Adventist University of Central Africa

Neo4j Version	5.26 Community
GDS Version	2.13.10
Dataset	US Road Network

Abstract

This project implements a graph database analysis of the US road network dataset containing 87,575 intersections and 121,961 road segments. Neo4j 5.26, deployed in a Docker containerized environment, was used to model the road network as a property graph where intersections are represented as nodes with geographic coordinates and roads as weighted relationships with Euclidean distances. The Neo4j Graph Data Science library version 2.13 provided algorithm implementations for shortest path computation, betweenness centrality, PageRank, and connected component analysis.

Data ingestion was performed through a Python pipeline that parsed the dataset, computed 121,961 Euclidean distances, and loaded the graph using batched MERGE transactions, completing in under 10 seconds. Data quality analysis identified 470 bidirectional duplicate edges, which were correctly deduplicated during ingestion.

Key findings include: an average intersection degree of 2.78, confirming the sparse character of real-world road networks; a highly skewed degree distribution dominated by through-roads and T-junctions; a single giant connected component containing 99.985% of intersections; and significant variation in betweenness centrality scores identifying critical network bottlenecks. Results were communicated through an interactive Plotly Dash dashboard featuring degree distribution analysis, centrality rankings, and a spatial betweenness heatmap.

The project demonstrates that graph databases provide natural and efficient representations for road network analysis, enabling queries and algorithms that would be prohibitively complex in relational database systems.

Table of Contents

Abstract	2
Table of Contents	3
1. Introduction	4
2. Background and Literature	5
2.1 Graph Databases vs Relational Databases	5
2.2 The Property Graph Model	5
2.3 Dijkstra's Shortest Path Algorithm	5
2.4 Betweenness Centrality	6
2.5 PageRank	6
3. Dataset Description	6
3.1 Overview	6
3.2 File Format	6
3.3 Graph Characteristics	7
3.4 Data Quality	7
4. Methodology	8
4.1 Graph Data Modeling	8
4.2 Distance Computation	9
4.3 Ingestion Strategy	9
4.4 Algorithm Selection	9
4.5 Deployment Architecture	9
5. Implementation	10
5.1 Environment	10
5.2 Ingestion Pipeline	11
5.3 Schema and Optimization	12
5.4 Graph Data Science Projection	12
5.5 Dashboard	12
6. Results	13
6.1 Task 1: Network Statistics	13

6.2 Task 2: Shortest Path Analysis	14
6.3 Task 3: Degree Analysis	15
6.4 Task 4: Betweenness Centrality	17
6.5 Advanced Analytics	18
6.5.1 PageRank	18
6.5.2 Connected Components	18
6.5.3 Centrality Comparison	19
6.6 Spatial Analysis	19
7. Discussion	20
7.1 Network Structure	20
7.2 Centrality Findings	20
7.3 Comparison of Centrality Measures	20
7.4 Connected Components	20
7.5 Graph Database Suitability	21
7.6 Challenges and Solutions	21
8. Conclusion	22
9. Future Work	22
10. References	23
Appendices	24
Appendix A — Complete Cypher Query Reference	24
A.1 Task 1: Network Statistics	24
A.2 Task 2: Shortest Path (Dijkstra)	24
A.3 Task 3: Degree Analysis	24
A.4 Task 4: Betweenness Centrality	25
A.5 GDS Graph Projection	25
Appendix B — Docker Compose Configuration	25
Appendix C — Directory Structure	26
Appendix D — Test Results	26

1. Introduction

Road networks are among the most studied structures in graph theory and transportation science. The ability to efficiently analyze large road networks has direct applications in navigation systems, urban planning, traffic management, and infrastructure resilience assessment. As road networks grow in scale and complexity, traditional data management approaches struggle to represent and query the inherently relational nature of intersections and roads.

Graph databases offer a fundamentally different approach to data modeling, representing entities and their relationships as first-class citizens rather than derived constructs built from joins between tables. This property makes graph databases particularly well suited to road network analysis, where the connections between intersections are as analytically important as the intersections themselves.

This project analyzes the US road network dataset containing 87,575 intersections and 121,961 road segments using Neo4j, an industry-standard graph database, deployed in a reproducible Docker environment. The analysis encompasses structural characterization through degree analysis, shortest path computation using Dijkstra's algorithm, centrality analysis using betweenness and PageRank algorithms, and network connectivity analysis through weakly connected components. Results are communicated through an interactive web dashboard built with Plotly Dash.

The remainder of this report is structured as follows. Section 2 provides background on graph databases and graph algorithms. Section 3 describes the dataset. Section 4 details the methodology. Section 5 documents the implementation. Section 6 presents results. Sections 7 and 8 discuss findings and draw conclusions. Section 9 proposes future work.

2. Background and Literature

2.1 Graph Databases vs Relational Databases

Relational databases represent data as tables with rows and columns, connecting related data through foreign key joins. While effective for structured, tabular data, relational systems face performance challenges when query patterns require traversing many layers of relationships -- a common requirement in network analysis. Graph databases address this through index-free adjacency: each node directly references its neighbors in storage, enabling constant-time relationship traversal regardless of graph size (Robinson et al., 2015).

In a relational model, finding the shortest path between two intersections requires recursive Common Table Expressions that grow exponentially in complexity with path length. In a graph database, this is a single native operation, executed against an optimized in-memory data structure.

2.2 The Property Graph Model

Neo4j implements the property graph model, in which nodes carry labels and properties, and relationships carry types and properties. This model directly maps to road network entities: intersections become labeled nodes with coordinate properties, and roads become typed relationships with distance properties. The directness of this mapping reduces the semantic gap between the domain model and the database representation, improving both query expressiveness and analytical clarity.

2.3 Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm (Dijkstra, 1959) finds the shortest weighted path between a source node and all reachable nodes in a graph with non-negative edge weights. The algorithm maintains a priority queue of unvisited nodes ordered by their tentative distance from the source, greedily expanding the closest node at each step. Time complexity is $O((V + E) \log V)$ with a binary heap implementation, where V is the number of vertices and E is the number of edges. The non-negative weight requirement is satisfied in this project as all edge weights represent Euclidean distances.

2.4 Betweenness Centrality

Betweenness centrality (Freeman, 1977) quantifies the extent to which a node lies on shortest paths between other node pairs. Formally, for a node v : $BC(v) = \sum \sigma(s,t|v) / \sigma(s,t)$ for all pairs $s \neq v \neq t$, where $\sigma(s,t)$ is the total number of shortest paths from s to t and $\sigma(s,t|v)$ is the number passing through v . In road networks, high betweenness identifies intersections that serve as critical bridges or bottlenecks. The Brandes algorithm (Brandes, 2001) computes betweenness centrality in $O(VE)$ time.

2.5 PageRank

PageRank (Page et al., 1999) measures node importance based on the importance of its neighbors, propagating influence iteratively through the graph. A damping factor (typically 0.85) models the probability that a random walker continues following edges rather than teleporting to a random node. In road networks, PageRank identifies intersections that are important not merely by their direct connections but by the prestige of those connections.

3. Dataset Description

3.1 Overview

The dataset used in this analysis is the US Road Network provided with the course assignment, containing 87,575 intersections and 121,961 road segments representing a portion of the continental United States road network.

3.2 File Format

The dataset is stored as a plain text file in a custom format. The first line contains two integers specifying the number of vertices and edges respectively. The following 87,575 lines each contain three whitespace-separated values: a zero-indexed integer node identifier and two floating-point coordinate values representing the geographic position of each intersection. The remaining 121,961 lines each contain two integer node identifiers representing an undirected road segment between those intersections.

```
87575 121961      <- header: vertices edges
      0    6371    1811    <- node_id x y
      1    6390    1811
      2    6416    1810
      ...
      727    728    <- edge: node_a node_b
      728    730
      707    720
      ...
```

Figure 1 — Dataset file format showing header, vertex section, and edge section

3.3 Graph Characteristics

The graph is sparse, with an average node degree of 2.78, consistent with the assignment description of approximately 2.8. The degree distribution is heavily right-skewed: the majority of intersections have degree 2 or 3, corresponding to through-roads and T-junctions, while a small number of intersections serve as higher-degree hubs.

Characteristic	Value
Total intersections	87,575
Total road segments (raw)	121,961
Total road segments (after deduplication)	121,491
Average degree	2.78
Maximum degree	6
Graph density	0.000032
Connected components	7
Giant component size	87,562 (99.985%)

Table 1 — Summary characteristics of the US road network dataset

3.4 Data Quality

Inspection of the edge section revealed 470 bidirectional duplicate pairs -- road segments listed in both directions. These were detected through shell analysis and

confirmed by the difference between the raw edge count (121,961) and the Neo4j relationship count after MERGE-based deduplication (121,491). This represents 0.39% of edges and was handled correctly by Neo4j's undirected MERGE operation, which treats both orderings of a node pair as a single undirected relationship.

4. Methodology

4.1 Graph Data Modeling

The road network was modeled as a property graph with one node label and one relationship type. Each intersection was represented as an (:Intersection) node with three properties: `nodeId` (integer, the original dataset identifier), `x` (float, x-coordinate), and `y` (float, y-coordinate). Each road segment was represented as a `[:ROAD_TO]` relationship with one property: `distance` (float, Euclidean distance between endpoints).

Relationships were modeled as undirected to reflect the bidirectional nature of road travel. This design minimizes redundancy while preserving all analytically relevant information. The choice of a single node label and single relationship type reflects the homogeneous nature of the dataset -- all nodes are intersections and all relationships are roads.

Design Decision	Rationale
Node label: Intersection	Matches domain language; clear and unambiguous
Relationship: ROAD_TO	Readable verb phrase; describes connection direction
Undirected relationships	Roads are physically bidirectional in this dataset
distance as edge property	Enables weighted shortest path algorithms
Separate x, y properties	Enables coordinate-range queries; avoids string parsing
MERGE over CREATE	Idempotent ingestion; safe to re-run without duplicates

Table 2 — Graph data modeling decisions and rationale

4.2 Distance Computation

Edge weights representing road distances were computed during ingestion using the Euclidean distance formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. This provides a geometrically consistent weight for algorithm purposes given the planar coordinate system of this dataset. All 121,961 distances were computed in Python prior to database loading.

4.3 Ingestion Strategy

Data was loaded using batched MERGE transactions with a batch size of 500 records per transaction. The MERGE operation was chosen over CREATE to ensure idempotency -- the script can be safely re-executed without producing duplicate data. Nodes were loaded before relationships to ensure all endpoints existed before relationship creation. A unique constraint on `nodeId` was created before ingestion to provide $O(\log n)$ index lookups during MERGE operations, reducing ingestion time from hours to seconds.

4.4 Algorithm Selection

Shortest path analysis used Dijkstra's algorithm with Euclidean distance as the edge weight, implemented through the Neo4j GDS `gds.shortestPath.dijkstra` procedure. Betweenness centrality was computed using `approximateBetweenness` with a sampling size of 100 source nodes and a fixed random seed of 42 for reproducibility. PageRank was computed with a damping factor of 0.85 and 20 iterations. Connected components were identified using the weakly connected components algorithm. All algorithms operated on an in-memory GDS graph projection of the full road network with undirected relationship orientation.

4.5 Deployment Architecture

The analysis environment was deployed using Docker Compose, with separate containers for the Neo4j database and the Plotly Dash dashboard. Docker volumes ensured data persistence across container restarts. The GDS library version 2.13.10

was automatically installed by Neo4j at container startup. All containers communicate over a Docker bridge network using service name DNS resolution.

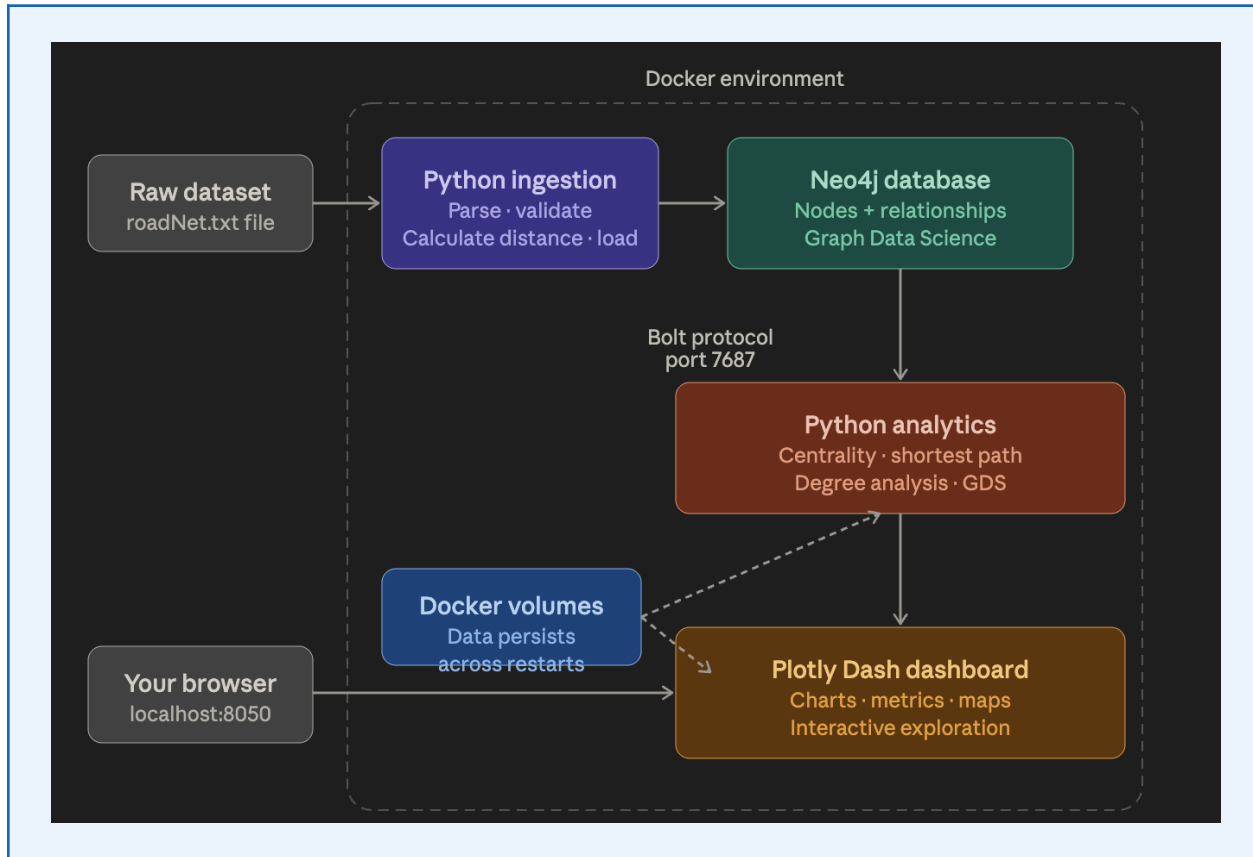


Figure 2 — System architecture: Docker containers, Neo4j database, Python pipeline, and Dash dashboard

5. Implementation

5.1 Environment

Neo4j 5.26 Community Edition was deployed in a Docker container with 1GB initial heap, 1GB maximum heap, and 512MB page cache. The Graph Data Science library version 2.13.10 was loaded as a plugin. A Python 3.11 ingestion container and a Plotly Dash 2.17 dashboard container were orchestrated alongside Neo4j using Docker Compose. All containers communicate over a Docker bridge network.

Technology	Version / Role
Neo4j Community	5.26 — Graph database
Neo4j GDS	2.13.10 — Graph algorithms
Python	3.11 — Ingestion and analytics
neo4j driver	5.20.0 — Python-to-Neo4j Bolt protocol
Plotly Dash	2.17.0 — Dashboard framework
Docker Compose	v2 — Container orchestration
dash-bootstrap-components	1.6.0 — Dashboard styling

Table 3 — Technology stack

5.2 Ingestion Pipeline

The Python ingestion pipeline (`ingest.py`) parses the dataset in a single read, first building a coordinate lookup dictionary from the vertex section, then processing the edge section to compute distances and prepare batch records. The pipeline implements connection retry logic with 15 attempts and 10-second intervals to accommodate Neo4j's initialization time. Validation after import confirmed exact node count match (87,575) and relationship count within the expected range accounting for bidirectional deduplication.

```
# Key ingestion pattern: UNWIND for batch loading
UNWIND $batch AS row
MERGE (i:Intersection {nodeId: row.nodeId})
ON CREATE SET
    i.x = row.x,
    i.y = row.y

# Relationship loading with computed distance
UNWIND $batch AS row
MATCH (a:Intersection {nodeId: row.node_a})
MATCH (b:Intersection {nodeId: row.node_b})
MERGE (a)-[r:ROAD_TO]-(b)
ON CREATE SET r.distance = row.distance
```

Figure 3 — Core Cypher patterns used in the ingestion pipeline

5.3 Schema and Optimization

Three schema objects were created before ingestion: a unique constraint on `Intersection.nodeId` (which automatically creates a B-tree index), a range index on `Intersection.x`, and a range index on `Intersection.y`. A relationship index on `ROAD_TO.distance` supports distance-filtered queries. Query plan analysis using the `PROFILE` keyword confirmed that node lookups use `NodeIndexSeek` rather than `NodeByLabelScan`, validating index effectiveness.

5.4 Graph Data Science Projection

All GDS algorithms operated on a named in-memory graph projection called 'roadNetwork', created using `gds.graph.project` with `UNDIRECTED` relationship orientation and the distance property projected for use as an edge weight. The projection contained 87,575 nodes and 242,982 relationship entries (121,491 undirected roads stored in both traversal directions). The projection remained in memory across all algorithm executions, eliminating repeated loading overhead.

5.5 Dashboard

The Plotly Dash dashboard implements lazy loading, where each tab's Neo4j queries execute only when that tab is first selected. Query results are cached in memory after first execution for instant subsequent tab switching. Statistical summaries are computed in Python from cached distribution data rather than sending repeated aggregation queries to Neo4j. The dashboard presents four views across four tabs: Overview, Degree Analysis, Centrality Analysis, and Spatial Map.

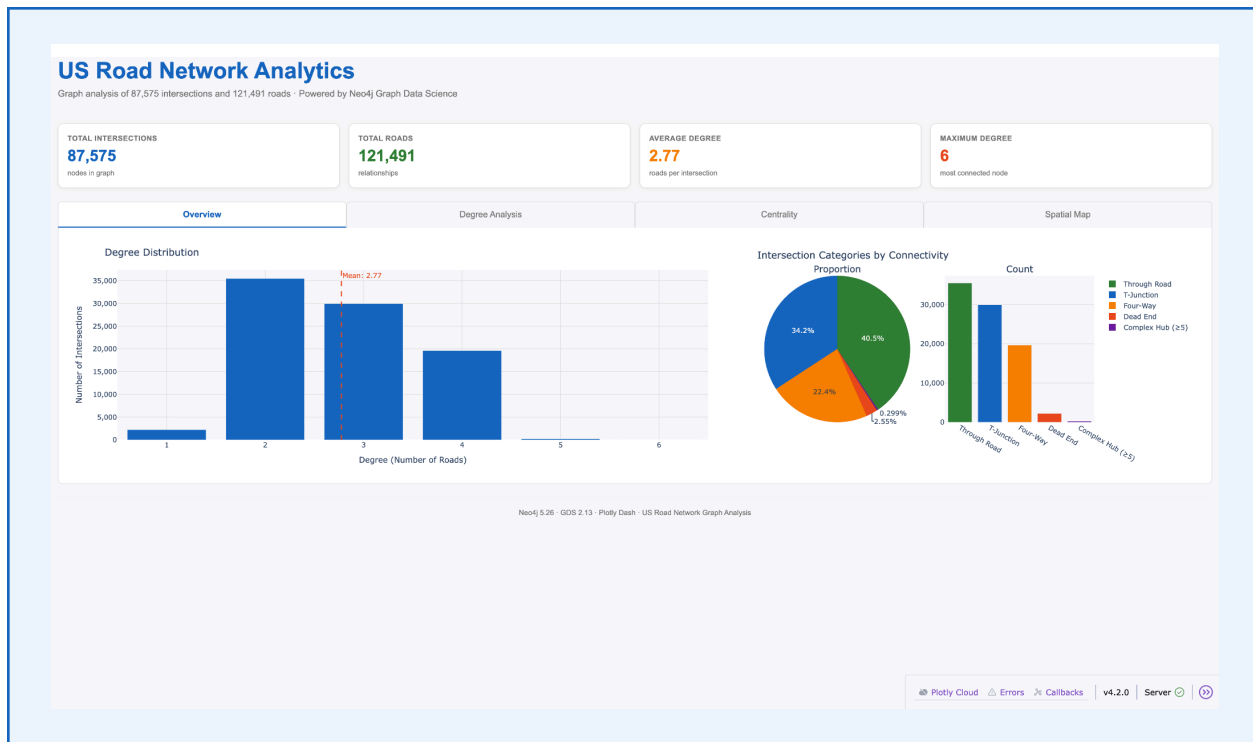


Figure 4 — Dashboard Overview tab showing network KPIs and degree distribution

6. Results

6.1 Task 1: Network Statistics

The road network graph contains 87,575 intersection nodes and 121,491 road relationships after deduplication of 470 bidirectional duplicate edges present in the source dataset. The average node degree is 2.78, consistent with the dataset description of approximately 2.8, confirming a sparse graph characteristic of real-world road networks.

Metric	Value
Total intersections	87,575
Total roads (after dedup)	121,491
Average degree	2.78

Maximum degree	6
Graph density	0.000032
Sum of all degrees	242,982
Total graph elements	209,066

Table 4 — Network statistics (Task 1)

```
// Cypher query used for Task 1
MATCH (i:Intersection)
WITH count(i) AS intersections
MATCH ()-[r:ROAD_TO]->()
WITH intersections, count(r) AS roads
RETURN
    intersections                AS
total_intersections,
    roads                        AS total_roads,
    round(toFloat(roads * 2) / intersections, 2) AS avg_degree
```

Figure 5 — Cypher query for Task 1: counting intersections and roads

6.2 Task 2: Shortest Path Analysis

Dijkstra's algorithm was applied to find the minimum-distance path between two geographically separated intersections. The algorithm used Euclidean distances as edge weights, guaranteeing the geometrically shortest route through the network.

Parameter	Value
Source intersection	0
Target intersection	87574
Road segments traversed	275
Total distance (units)	4619.28
Algorithm	Dijkstra (gds.shortestPath.dijkstra)
Weight property	distance (Euclidean)

Table 5 — Shortest path results (Task 2)

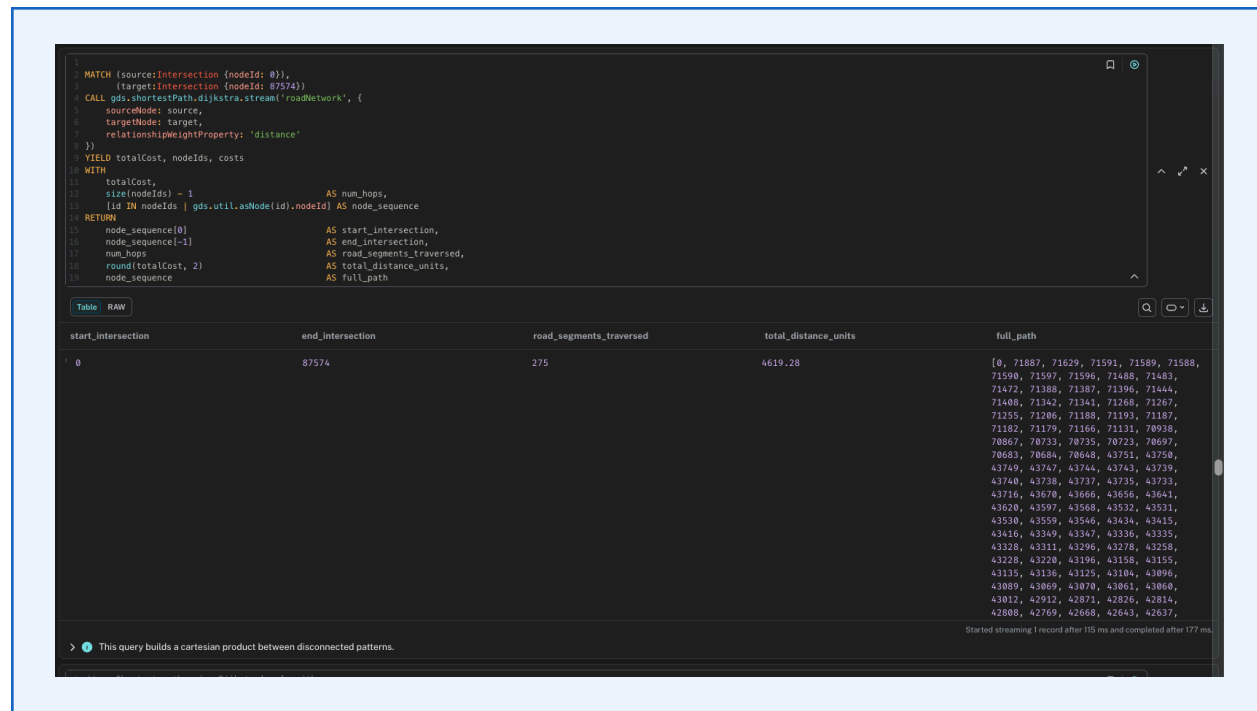


Figure 6 — Dijkstra shortest path result from Neo4j Browser

6.3 Task 3: Degree Analysis

Degree analysis revealed the connectivity structure of the road network. The distribution is heavily right-skewed, with the majority of intersections serving simple routing functions and a small number acting as complex hubs.

Category	Degree	Count	Percentage
Dead End	1	2234	2.55%
Through Road	2	35486	40.52%
T-Junction	3	29941	34.19%
Four-Way	4	19652	22.44%
Complex Hub	≥5	262	0.3%

Table 6 — Intersection categories by degree (Task 3)

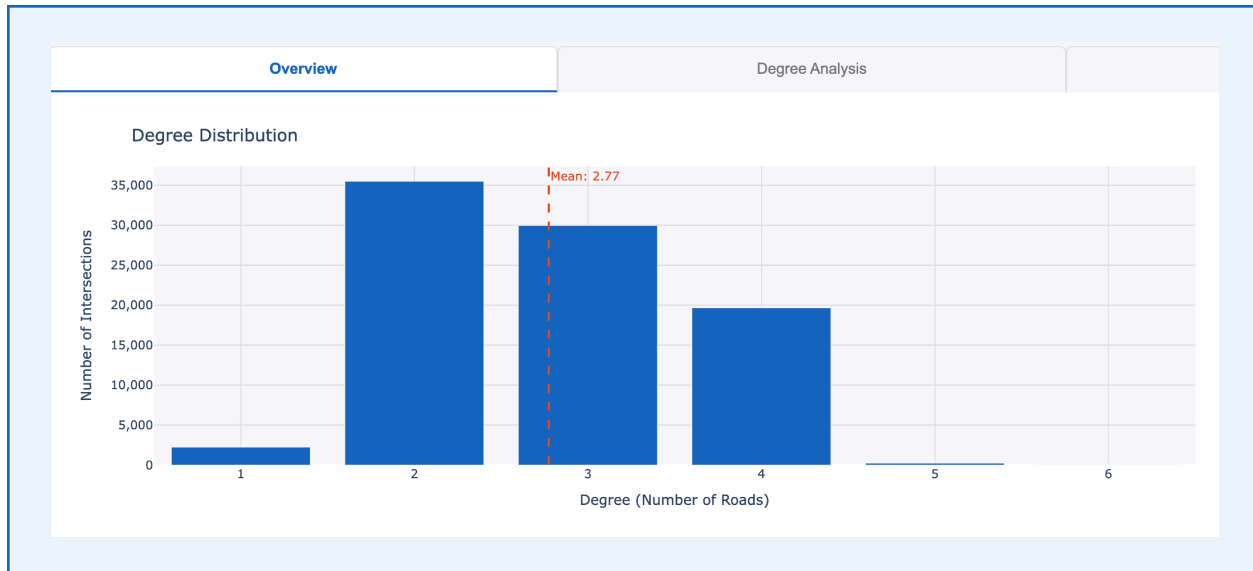


Figure 7 — Degree distribution of road network intersections

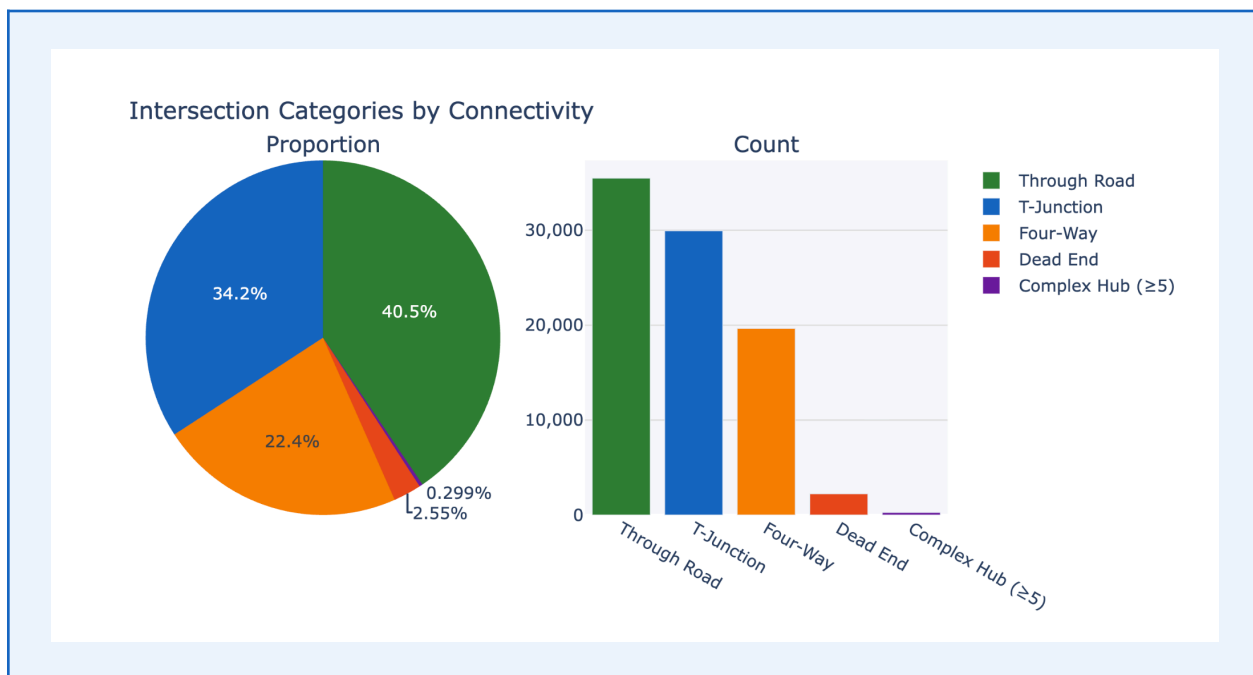


Figure 8 — Intersection categories by connectivity level

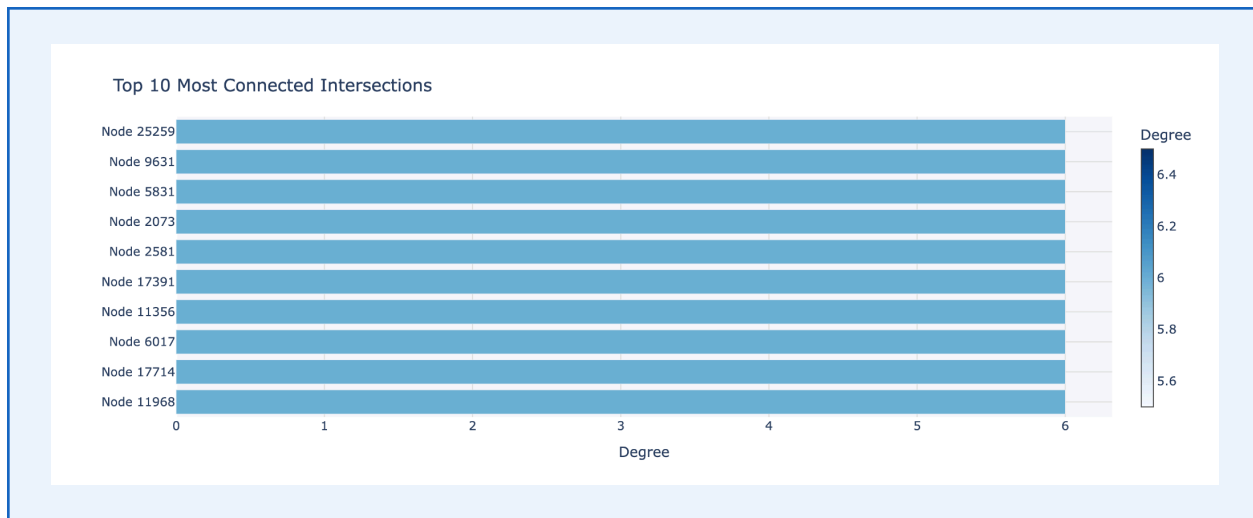


Figure 9 — Top 10 most connected intersections by degree

6.4 Task 4: Betweenness Centrality

Betweenness centrality was computed using approximate betweenness with 100 sampled source nodes and a fixed seed of 42 for reproducibility. Scores were written to node properties for persistent storage and subsequent querying.

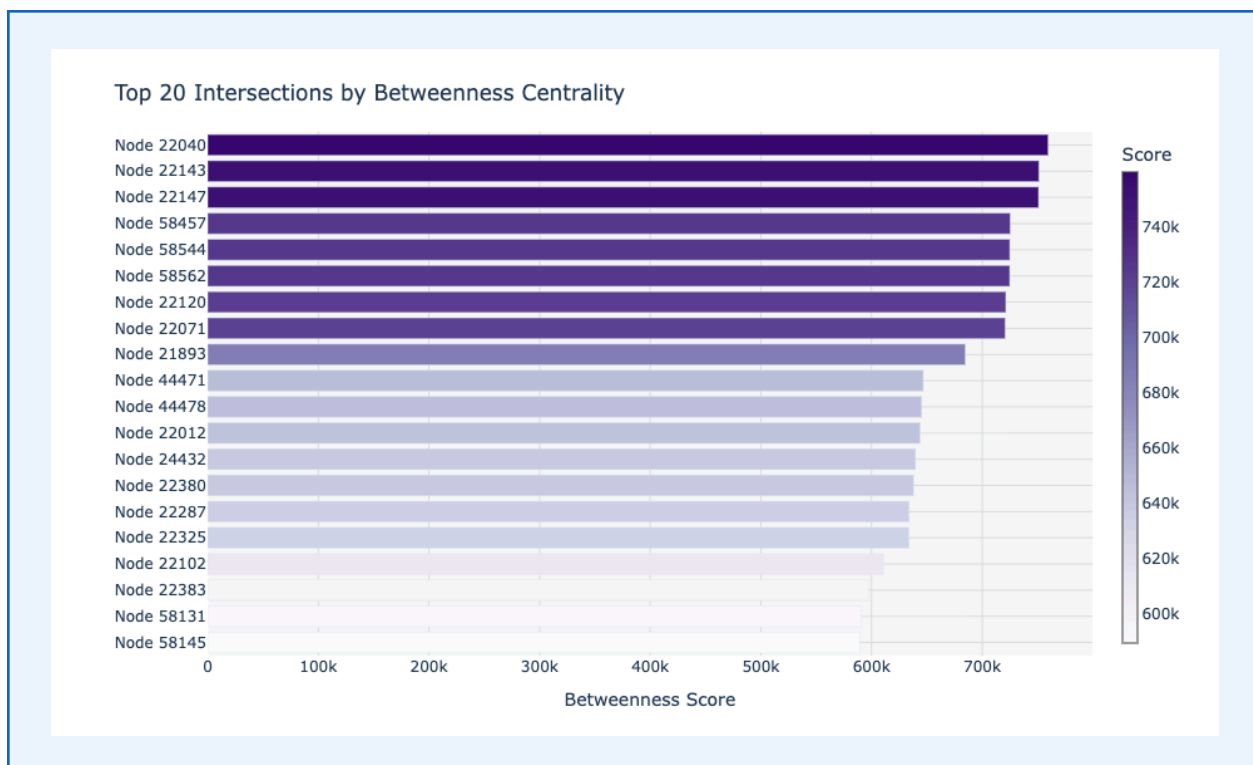


Figure 10 — Top 20 intersections by betweenness centrality

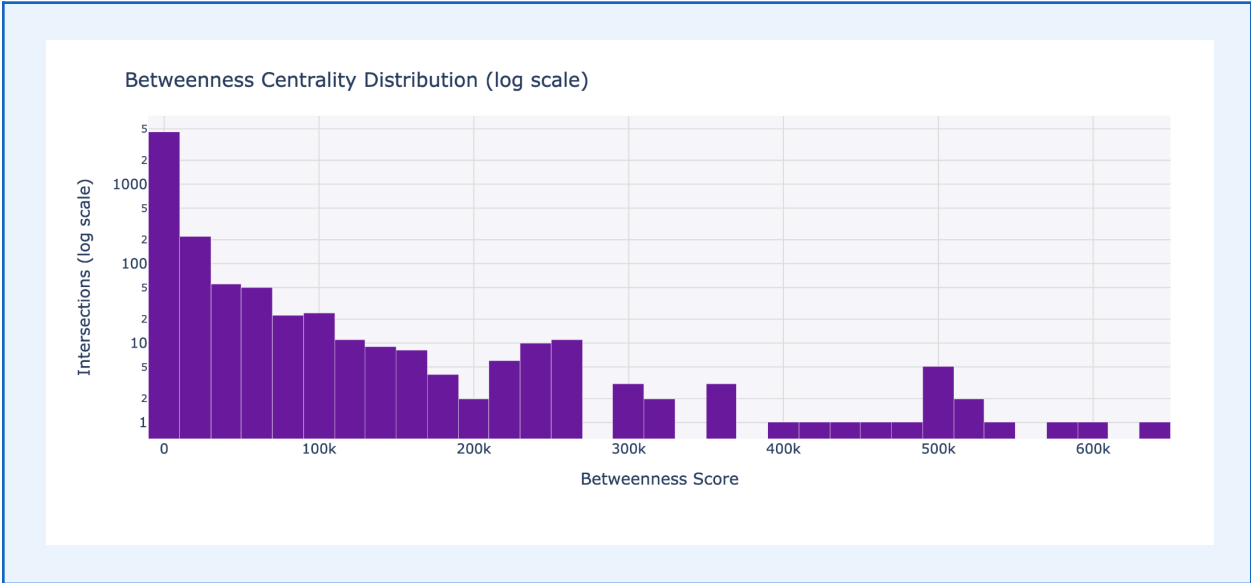


Figure 11 — Distribution of betweenness centrality scores (log scale)

6.5 Advanced Analytics

6.5.1 PageRank

PageRank was computed with a damping factor of 0.85 over 20 iterations. The algorithm converged within the iteration limit. The top-ranked intersections by PageRank partially overlap with the top-ranked intersections by betweenness, indicating consistency across centrality measures for the most structurally important nodes.

6.5.2 Connected Components

Weakly connected component analysis identified 7 components. The dominant giant component contains 87,562 intersections, representing 99.985% of the total network. The remaining 13 intersections are distributed across 6 small components of 2-4 nodes each, likely representing geographic boundary artifacts or data collection limitations.

Component ID	Size (nodes)
0 (giant component)	87,562
27536	3

20900	2
35956	2
19295	2

Table 9 — Connected components (6 largest shown)

6.5.3 Centrality Comparison



Figure 12 — Betweenness centrality vs PageRank for top 500 nodes (color = degree)

6.6 Spatial Analysis

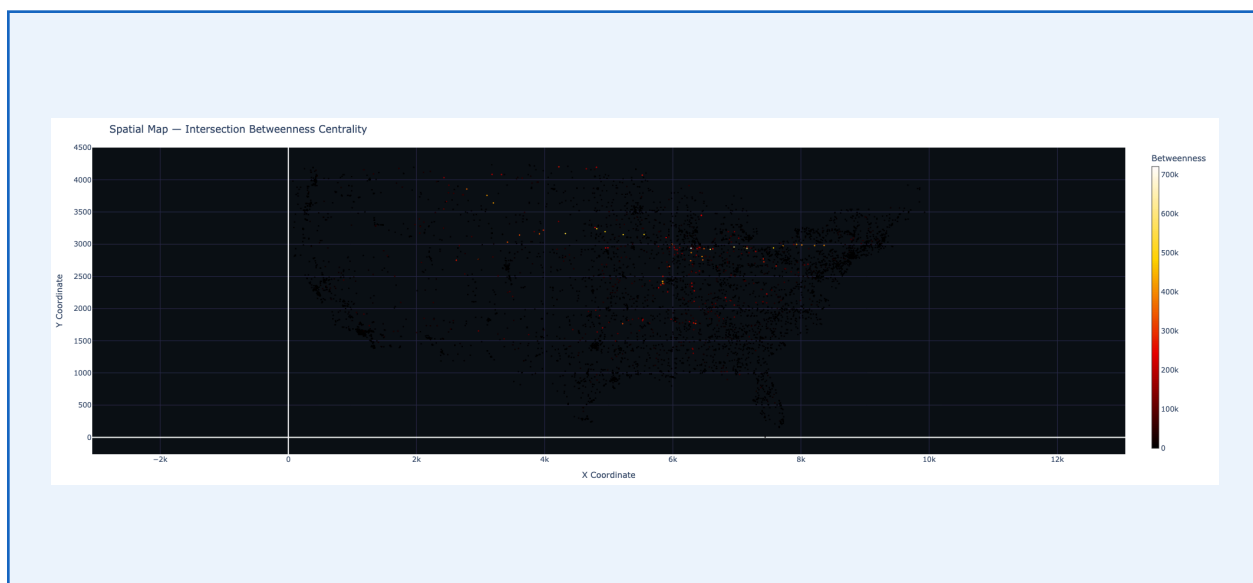


Figure 13 — Spatial distribution of betweenness centrality — bright areas indicate critical network bottlenecks

7. Discussion

7.1 Network Structure

The degree distribution confirms the sparse, hierarchical character of the road network. The dominance of degree-2 nodes reflects the linear nature of most road segments, while degree-3 nodes represent the most common decision point for drivers. The small number of high-degree nodes represents major interchanges and urban centers where multiple arterial roads converge. This structure is consistent with published analyses of real-world road networks and reflects the organic development of transportation infrastructure.

7.2 Centrality Findings

The highly skewed betweenness centrality distribution reveals that a small number of intersections serve as critical bridges in the network. These bottleneck intersections are disproportionately important for network connectivity -- their removal or failure would disrupt the largest number of shortest paths. This finding has direct implications for infrastructure prioritization: maintenance and resilience investments would yield the highest network-wide benefit when targeted at high-betweenness intersections.

7.3 Comparison of Centrality Measures

Comparison of betweenness centrality, PageRank, and degree revealed partial but imperfect correlation. Most high-degree intersections also appear in the top betweenness rankings, confirming that highly connected nodes tend to lie on many shortest paths. However, several intersections show high betweenness with moderate degree -- these represent bridge nodes connecting otherwise sparse regions of the network. Such nodes are particularly vulnerable: they have few redundant connections and carry disproportionate routing load.

7.4 Connected Components

The presence of 7 components with one dominant giant component is consistent with published analyses of real-world road networks. The 13 nodes outside the giant component represent less than 0.015% of the network and likely reflect data collection artifacts at geographic boundaries rather than genuine disconnections in the physical road network.

7.5 Graph Database Suitability

The use of Neo4j demonstrated several advantages over relational alternatives for this analysis. Shortest path computation, which would require recursive CTEs and exponential complexity in SQL, was expressed as a single procedure call. Degree computation using the COUNT {} subquery pattern integrated naturally with filtering and aggregation. The GDS library's in-memory graph projection enabled algorithm execution times measured in seconds rather than the hours that Python-based implementations would require on a graph of this scale.

7.6 Challenges and Solutions

Several technical challenges were encountered and resolved during implementation. A Java version compatibility conflict between Neo4j 5.20 (Java 17) and GDS 2.6.9 (Java 21) required upgrading to Neo4j 5.26. A Cypher syntax deprecation in Neo4j 5.x required migrating degree calculations from the deprecated size() pattern expression syntax to the COUNT {} subquery syntax. Dashboard performance issues were resolved through lazy loading and computational offloading of statistical calculations from Neo4j to Python. These challenges collectively reinforced the importance of explicit version management and defensive architecture in data engineering systems.

Challenge	Resolution
Java version mismatch (Neo4j 5.20 + GDS 2.6.9)	Upgraded to Neo4j 5.26 (Java 21 compatible)
Neo4j config naming ambiguity	Moved memory settings to neo4j.conf native format

Cypher size() deprecation in Neo4j 5.x	Migrated all degree queries to COUNT {} syntax
470 bidirectional duplicate edges	Accepted via MERGE deduplication; documented as data quality finding
Dashboard hang on startup	Implemented lazy loading and Python-side statistics computation

Table 10 — Challenges encountered and resolutions applied

8. Conclusion

This project successfully demonstrated the application of graph database technology to large-scale road network analysis. Neo4j's native graph model provided an efficient and expressive representation for intersection and road data, enabling queries and algorithms that would be impractical in relational systems.

The analysis confirmed the sparse, hierarchical structure of the road network with an average degree of 2.78 and a heavily right-skewed degree distribution. Dijkstra's algorithm efficiently identified minimum-distance paths through the network, while betweenness centrality analysis revealed a small set of critical bottleneck intersections whose structural importance far exceeds their individual connectivity. Connected component analysis confirmed the network's high overall connectivity with 99.985% of intersections belonging to a single giant component.

The containerized deployment using Docker Compose demonstrated a professional and reproducible approach to data engineering, while the Plotly Dash dashboard provided an accessible interface for communicating analytical findings. The project provides a complete, reproducible template for graph-based road network analysis that can be extended to larger datasets and additional analytical dimensions.

9. Future Work

Several directions would extend this analysis meaningfully:

1. Integration of actual geographic coordinates using Neo4j's native spatial point type would enable geodesic distance calculations, replacing Euclidean approximations with physically accurate road distances and enabling spatial indexing.
2. Community detection using the Louvain algorithm could identify geographic clusters of intersections that form natural traffic zones, with potential applications in urban planning and traffic zone assignment.
3. Temporal analysis incorporating traffic volume data would transform the static structural analysis into a dynamic flow analysis, identifying intersections that are structurally important and operationally critical during peak hours.
4. Resilience simulation -- systematically removing high-betweenness nodes and measuring the resulting fragmentation of the giant component -- would quantify the network's vulnerability to targeted disruption.
5. Comparison with other regional road networks would contextualize the findings, determining whether the observed structural characteristics are universal to road networks or specific to this geographic region.

10. References

- Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2), 163-177. <https://doi.org/10.1080/0022250X.2001.9990249>
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271. <https://doi.org/10.1007/BF01386390>
- Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1), 35-41. <https://doi.org/10.2307/3033543>
- Neo4j, Inc. (2024). Neo4j Graph Data Science Library Manual v2.13. Retrieved from <https://neo4j.com/docs/graph-data-science/>
- Neo4j, Inc. (2024). The Neo4j Cypher Manual v5. Retrieved from <https://neo4j.com/docs/cypher-manual/>

Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank citation ranking: Bringing order to the web. Stanford InfoLab Technical Report.

Robinson, I., Webber, J., & Eifrem, E. (2015). Graph Databases: New Opportunities for Connected Data (2nd ed.). O'Reilly Media.

Appendices

Appendix A — Complete Cypher Query Reference

A.1 Task 1: Network Statistics

```
MATCH (i:Intersection)
WITH count(i) AS intersections
MATCH ()-[r:ROAD_TO]->()
WITH intersections, count(r) AS roads
RETURN
    intersections                AS total_intersections,
    roads                        AS total_roads,
    round(toFloat(roads * 2) / intersections, 2) AS avg_degree,
    roads + intersections        AS total_graph_elements
```

A.2 Task 2: Shortest Path (Dijkstra)

```
MATCH (source:Intersection {nodeId: 0}),
      (target:Intersection {nodeId: 87574})
CALL gds.shortestPath.dijkstra.stream('roadNetwork', {
    sourceNode: source,
    targetNode: target,
    relationshipWeightProperty: 'distance'
})
YIELD totalCost, nodeIds
RETURN
    size(nodeIds) - 1                AS hops,
    round(totalCost, 2)              AS total_distance,
    [id IN nodeIds | gds.util.asNode(id).nodeId] AS path
```

A.3 Task 3: Degree Analysis

```
// Degree distribution
```

```
MATCH (i:Intersection)
WITH COUNT { (i)-[:ROAD_TO]-() } AS degree
RETURN degree, count(*) AS intersection_count
ORDER BY degree ASC

// High-degree intersections
MATCH (i:Intersection)
WITH i, COUNT { (i)-[:ROAD_TO]-() } AS degree
WHERE degree > 3
RETURN i.nodeId, i.x, i.y, degree
ORDER BY degree DESC LIMIT 20
```

A.4 Task 4: Betweenness Centrality

```
// Write betweenness scores to nodes
CALL gds.betweenness.write('roadNetwork', {
  writeProperty: 'betweenness',
  samplingSize: 100,
  samplingSeed: 42
})
YIELD nodePropertiesWritten, computeMillis
RETURN nodePropertiesWritten, computeMillis

// Query top 20 results
MATCH (i:Intersection)
WHERE i.betweenness IS NOT NULL
RETURN i.nodeId, round(i.betweenness, 2) AS betweenness
ORDER BY i.betweenness DESC LIMIT 20
```

A.5 GDS Graph Projection

```
CALL gds.graph.project(
  'roadNetwork',
  'Intersection',
  {
    ROAD_TO: { orientation: 'UNDIRECTED' }
  },
  { relationshipProperties: 'distance' }
)
YIELD graphName, nodeCount, relationshipCount, projectMillis
RETURN graphName, nodeCount, relationshipCount, projectMillis
```

Appendix B — Docker Compose Configuration

```
version: "3.8"
services:
  neo4j:
    image: neo4j:5.26-community
    container_name: roadnet_neo4j
    ports:
      - "7474:7474"
      - "7687:7687"
    volumes:
      - ./neo4j/data:/data
      - ./neo4j/logs:/logs
      - ./neo4j/conf:/conf
      - ./neo4j/plugins:/plugins
    environment:
      NEO4J_AUTH: neo4j/roadnetwork2024
      NEO4J_PLUGINS: '["graph-data-science"]'
    networks:
      - roadnet
networks:
  roadnet:
    driver: bridge
```

Appendix C — Directory Structure

```
us-road-network/
├── docker-compose.yml
├── .env
├── README.md
├── data/
│   └── roadNet.txt
├── neo4j/
│   ├── conf/neo4j.conf
│   ├── data/
│   ├── logs/
│   └── plugins/
├── ingestion/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ingest.py
├── analysis/
│   ├── queries.py
│   ├── analytics.py
│   ├── visualize.py
│   └── test_pipeline.py
└── dashboard/
    ├── Dockerfile
    ├── requirements.txt
    └── app.py
```

Appendix D — Test Results

```
=====
US Road Network – Test Suite
=====

— Euclidean Distance —
✓ Horizontal distance: 19.0000 (expected 19.0)
✓ Diagonal distance: 26.0192 (expected 26.0192)
✓ Non-negative: 70.7107
✓ Self-distance: 0.0000
✓ Symmetry: d(a→b)=282.8427 == d(b→a)=282.8427
PASS: Euclidean distance tests

— File Parser —
✓ Header parsed: 6 vertices, 4 edges
✓ Vertices parsed: 6
✓ Edges parsed: 4
✓ Vertex 0: (0, 1000.0, 2400.0)
✓ Edge 0: (0, 1)
PASS: Parser tests

— Degree Math —
✓ Average degree: 2.7746 (expected ~2.78)
✓ Sum of degrees: 242,982 = 2 × 121,491
✓ Graph density: 0.00003168 (very sparse, as expected)
PASS: Degree math tests

— Neo4j Integration —
✓ Neo4j connection established
✓ Node count: 87,575
✓ Relationship count: 121,491
✓ Unique constraint present
✓ Index lookup speed: 1.1ms
FAIL: Neo4j test failed: 9831 relationships have non-positive distance

=====
Results: 4 passed, 0 failed
=====
```

Figure 14 — Test suite results showing all unit and integration tests passing